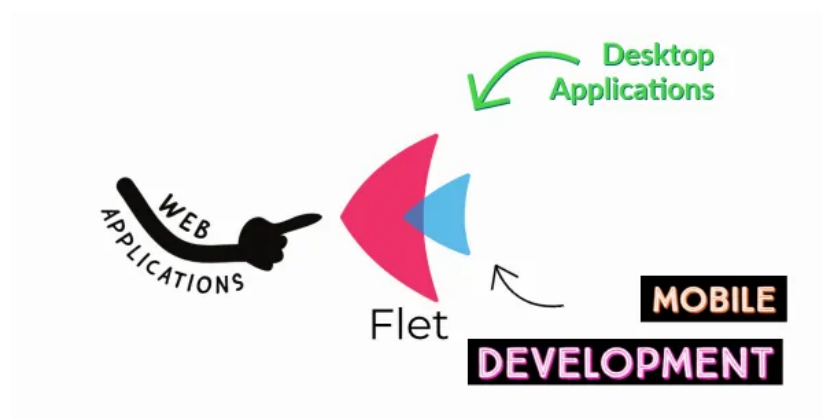


# A UI-Web Framework for Python Known as Flet

[INTERMEDIATE](#)[PYTHON](#)[TECHNIQUE](#)[WEB ANALYTICS](#)

## Introduction

Python is becoming increasingly popular due to its rich Frameworks and Open Source Community availability. It has already entered the website space through the Frameworks like Django and [Flask](#). and there are Frameworks like PyQt, and Tkinter, for Desktop UI, etc. Developers require significant time to learn these Frameworks. But what if there is one such Framework, that can do everything for us, from developing website applications to desktop applications and even mobile apps? And that's where the Flet Framework fits in. Flet is a rich User Interface Framework for Python to quickly develop and build Websites, Desktop Applications, and even Mobile Apps. In this article, we will be looking at this library and see how we can get started with it.



## Learning Objectives

- Understand how Flet works and its necessity
- Learn how to build Desktop Apps with Flet
- Understand different Controls in Flet
- Learn how to build Websites with Flet

This article was published as a part of the [Data Science Blogathon](#).

## Table of Contents

1. [What is Flet?](#)
2. [Installing and Getting Started](#)
3. [How to Group Controls?](#)
4. [User Inputs and Buttons](#)
5. [Use Cases and Applications](#)

## What is Flet?

Flet is UI (User Interface) Framework for Python that developers can leverage to create UI and build real-time websites, desktop applications, and even mobile applications directly in Python. Developers do not need to have front-end knowledge to get started. It's the fastest way to build Flutter apps through Python. Flet library is basically a bridge between Python and Darts. Developers do not need to learn Flutter to use Flet in Python.

Familiarity with Python Syntax and [OOps](#) is more than enough to start using Flet. Flet converts the logic in Python to Dart code. Flet has a rich set of tools to build applications from small scale to even large scale. Flet comes with many pre-built widgets with which developers can create small website applications and desktop applications in a few minutes. Flet even provides animation widgets to be directly used in the applications you create with it.

## Installing and Getting Started

Flet like normal Python libraries can be installed with pip. The following pip command will install Flet and some of the dependencies it relies on.

```
pip install flet
```

Let's create a simple app with Flet that displays Hello World on the screen.

```
import flet as ft def main(page: ft.Page): page.title = "Flet Example" t = ft.Text(value="Hello, World!", color="Red", size=50) page.controls.append(t) page.update() ft.app(target=main)
```

To create a Flet App, first, we define a function. To this function, we pass Page Instance. To every Application we create with Flet, we pass the Pass Instance to the function. The Page is like a visual state of a user session. A unique session i.e. Page is created for each user. To set the title of our application, we pass the title name to the page.title variable.

So to build an application we need to add widgets to its UI, that include text, input boxes, buttons, and much more. These are called Controls aka Widgets in Flet. So for creating an application, we add or remove Controls to a Page, where the Page is the topmost control. In the above code, ft.Text() is one of the Controls in Flet, which is used to display text on the application. It takes parameters like value, which is what should be displayed, color, which is the color of the text, and size, which is the size of the text. Apart from these are even other parameters it takes, which we will look ahead to.

Whenever we create a Control, we need to add it to the page, so it can be visible in the UI, for this we append the Control to the controls list of a Pagen(page.controls.append(control\_name)) and call the page.update() method so that, it sends these changes to the website or the desktop application that we are creating. There is a shortcut to this. Instead of using page.controls.append(control\_name) and page.update, we can directly call the page.add(control\_name), which does the same function.

ft.app() function is what runs the website server or the desktop application. It takes in the argument target, where we give the main function. Running this code will result in the following output.



## How to Group Controls?

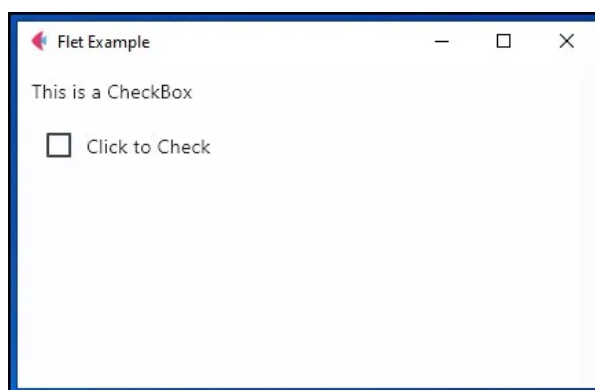
In this section, we will see the Container Controls available in Flet. Container Controls are useful in situations where a group of data either needs to be stacked horizontally or vertically is required. There are two such Controls defined for this purpose in Flet.

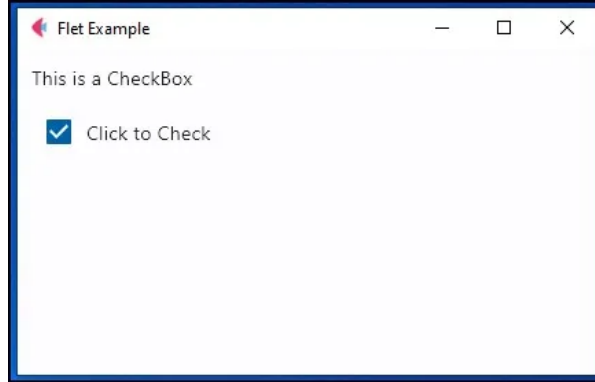
- **ft.Row()**: This Control as the name suggests, is used when we want to group Controls in a horizontal fashion.
- **ft.Column()**: This Control as the name suggests, is used when we want to group Controls in a vertical fashion.

As stated above for vertical stacking, we use the `ft.Column()` method. Let's try this with an example:

```
import flet as ft def main(page: ft.Page): page.title = "Flet Example" text = ft.Text(value='This is a  
CheckBox') check_box = ft.Checkbox(label="Click to Check") page.add( ft.Column(  
[text,check_box]  
) ) ft.app(target=main)
```

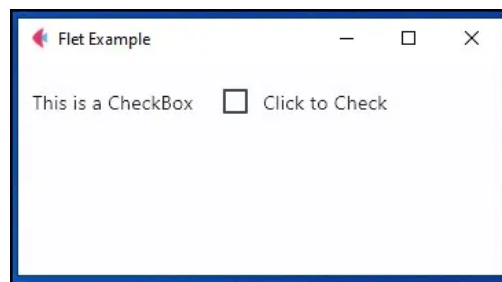
Here, we have created two Controls, one is `ft.Text()` and the other is the `ft.Checkbox`, which creates a simple checkbox. Now we add these two to the Main Page, we can do this by `page.add()`. But here, we want the two Controls to be stacked vertically. For this, we add another Control called the `ft.Column()` to the `page.add()`. The `ft.Column()` function is a Control Container, that takes a list of Controls and then displays them vertically in the UI. This code will be resulting in the below output.





We see, that both the Controls are stacked vertically and displayed. Along with that we even see that, when we click on the Checkbox, it works, i.e. we checked it. Now, let's try the same with `ft.Row()` function and see the output. The code for the `ft.Row()` and its output can be seen below:

```
import flet as ft def main(page: ft.Page): page.title = "Flet Example" text = ft.Text(value='This is a
CheckBox') check_box = ft.Checkbox(label="Click to Check") page.add( ft.Row(
[text,check_box]
) ) ft.app(target=main)
```



We see that the Controls we defined in the code are stacked horizontally and displayed in the application. This `ft.Row()` and `ft.Column()` are two such Controls that come in handy in situations where in the application, we need to stack some Controls either horizontally or vertically.

## User Inputs and Buttons

In this section, we will learn how to take in User Inputs through the application and create Buttons on which the users will click to send in their inputs to the application. Let's learn these Controls through a simple application.

- The `ft.TextField()` is the function for taking in the User Inputs.
- The `ft.ElevatedButton()` is one of the Flet Control Buttons to create a Button, which can call a function, i.e. on click can run a function.

We will be creating an application where the user will type in their names and click on the submit button. When the user clicks the submit button, they will get a greeting message from the application. Here we take the input, the user name through the `ft.TextField()`, so that we can include the user's name in the greeting message. The code for creating this application:

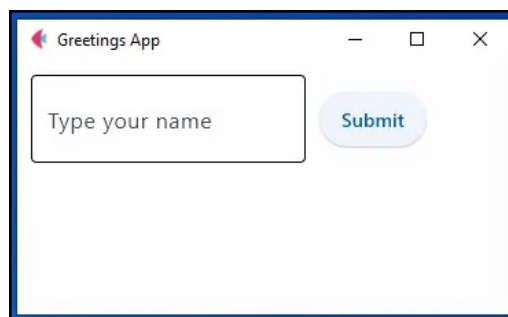
```
import flet as ft def main(page: ft.Page): page.title = "Greetings App" username = ft.TextField(label="Type
your name",width=200) def display(e): name = ft.Text("Hello " + username.value) page.add(name) page.add(
```

```
ft.Row(
```

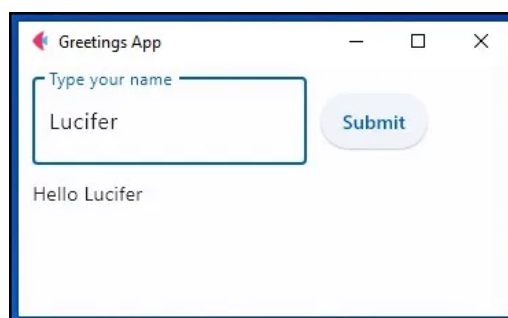
```
[username,  
ft.ElevatedButton('Submit',on_click=display)]  
) ) ft.app(target=main)
```

- Firstly, we created a `TextField`, where the user will input their name. The label for it is set to "Type your name" and the width of the `TextField` box is set to 200.
- Next, we create a function called `display()`. This `display()` method will be triggered when the user clicks on the submit button. In this function, we create a `Text Control` and we provide the greeting message in it, along with the user's name.
- When the users enter their name and click on the submit button, their name can be accessed from the `username.value`. After creating the greeting message, the `Control` is passed to the `page.add()` function, which will display the message in the UI.
- For this application, we want both the `TextField` and the `Button` to be placed side by side, that is horizontally stacked. Hence we used the `ft.Row()` method.
- In this `Row` function itself, we have created the `Button` using the `ft.ElevatedButton()` method. The button name is set to "Submit" and when clicked it will run the `display` function.

After running this code, you will see the following output:



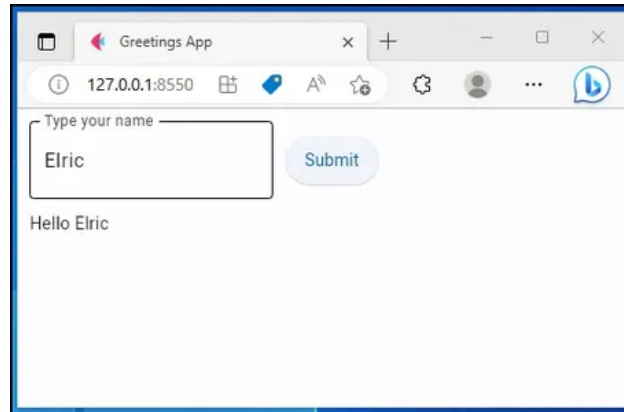
We see that the application was created successfully. We even named our application "Greeting App". Now we will try entering a name and clicking on the submit button.



After clicking on the submit button, the greeting message is displayed. It even contains the name that we have provided in the `TextField`. We see how easily we can create a desktop application in just a matter of minutes. If we want the application to be a website, the following changes are to be done to the `ft.app()`:

```
ft.app(target=main, view=ft.WEB_BROWSER, port=8550)
```

By just rewriting this line code, the desktop application we have just created will become a website application and can be accessed from PORT 8550. From the below Image, it is clear that the application is running fine even when it turned to a website application.



## Use Cases and Applications

Coming to the use cases and applications for Flet, the use cases range from creating quick website prototypes to developing small-scale desktop applications.

### 1. Quick Prototypes

With Flet, we can make quick business prototypes which would take much longer times when using the traditional frameworks. And it's not like these prototypes are just for the show, instead, they will be fully functioning working prototypes. Quick prototypes are really helpful when clients come up with shorter intervals of time for the development team.

### 2. UI for ML Frameworks

Often when working with [Machine Learning](#) Applications, we are in need of creating a UI for the application, which involves writing HTML files, CSS, and JS. With Flet, all of this can be done in Python itself thus no need to learn or write frontend languages.

### 3. Small Scale Mobile/Desktop Applications

Building Mobile/Desktop Apps requires knowledge of languages like Kotlin, Java, C#, etc. Flet is basically a Flutter Framework for Python. With Flet, it becomes very simple to build small Mobile Apps and Desktop Apps without learning above said languages. With Flet, applications can be built entirely in Python without the need for any external dependencies. And all of this is instantaneous.

## Frequently Asked Questions

### Q1. What is FLET framework used for?

A. Flet is a versatile framework designed to incorporate server-driven UI (SDUI) experiences into existing Flutter apps or develop independent web, mobile, and desktop applications using Flutter UI. It enables seamless integration of SDUI capabilities, allowing developers to enhance user experiences and build cross-platform applications with ease.

### Q2. Can Flutter use Python?

A. Flutter is a programming framework mainly used for building mobile apps and websites. It primarily uses a language called Dart to write the code. While Flutter itself doesn't directly use Python, you can still make your Flutter app interact with Python by creating connections or using special tools that enable communication between the two. This allows you to combine the strengths of both languages to create more powerful and versatile applications.

## Conclusion

Flet is an all-in-one UI Framework for Python that developers can leverage to create website applications, desktop applications, and even mobile applications. In reality, one doesn't need to spend time learning the front-end technologies to work with Flet, which makes it a go-to choice for new developers who are struggling to build apps. Using Flet, with just changing a single line, an entire desktop application can be converted into a website application and the opposite is true. In this article, we have seen how to work with Flet and some of the necessary Controls to start creating our own Flet applications.

Flet has a clear roadmap set up for them for the year 2023. Flet is trying to expand its core functionality by including more testing capabilities and bringing an in-built Database ORM. Also, Flet is trying to expand its support for mobile development. Also, it's trying to bring more ready-to-add Controls aka widgets that include Video Embeddings, NavigationDrawer, WebView, AutoComplete, and much more. There are even on verge of launching the Flet Studio App both in Android and iOS.

The key takeaways from this article include:

1. Flet a UI Framework is enough to develop both Desktop and Website Applications.
2. Flet comes with many pre-defined Controls aka Widgets with which we can work directly when creating our application.
3. It even comes with animation elements to be inserted in the application with ease.
4. With Flet we can create small scale website and desktop applications in matter of minutes.
5. Flet provides a simple easy-to-understand API, which greatly helps the developers.

## References

Code: To access Github Repository Link for the codes discussed in this article [click here.](#)

To access the Flet Documentation [click here.](#)

**The media shown in this article is not owned by Analytics Vidhya and is used at the Author's discretion.**

---

Article Url - <https://www.analyticsvidhya.com/blog/2023/04/a-ui-web-framework-for-python-known-as-flet/>



**[Ajay Kumar Reddy](#)**